

Міністерство освіти і науки України
Карпатський національний університет ім. Василя Стефаника

кафедра КНІС

Лабораторна робота № 3
з дисципліни “Теорія програмних систем”
“ПОБУДОВА ДІАГРАМ КЛАСІВ”

виконав:
студент групи ІСТ-31
Лопуляк В. І.

перевірила:
Штаєр Л.О.

Івано-Франківськ

2026

Мета роботи: одержати практичні навички у побудові діаграм класів для обраного варіанту комп'ютерної системи. **Обладнання:** ПК з доступом до мережі інтернет, браузер, онлайн-редактор PlantUML.

Завдання: Варіант 25. Побудувати діаграму класів для "Електронної системи обліку робочого часу працівника на підприємстві".

Хід роботи

1. На основі опису предметної області було виділено 11 логічно пов'язаних класів системи.
2. Для кожного класу визначено необхідні атрибути та операції (не менше 5 для кожного) із зазначенням типів даних (Int, String, Float, Date, DateTime, Boolean).
3. Класи було пов'язано між собою з використанням усіх типів відношень:
 - Наслідування: абстрактний клас Користувач є базовим для класів Працівник та Адміністратор.
 - Агрегація: Відділ містить працівників (час життя об'єкта Працівник не залежить від відділу).
 - Композиція: Працівник включає об'єкт КарткаДоступу (жорстка прив'язка до профілю), а СистемаОбліку складається з терміналів.
 - Асоціація: зв'язки між працівником, його графіком роботи та записами обліку часу.
 - Інстанціювання (залежність): ТерміналПропуску створює об'єкти ЗаписОблікуЧасу.

4. Побудова діаграми здійснювалась у середовищі PlantUML.

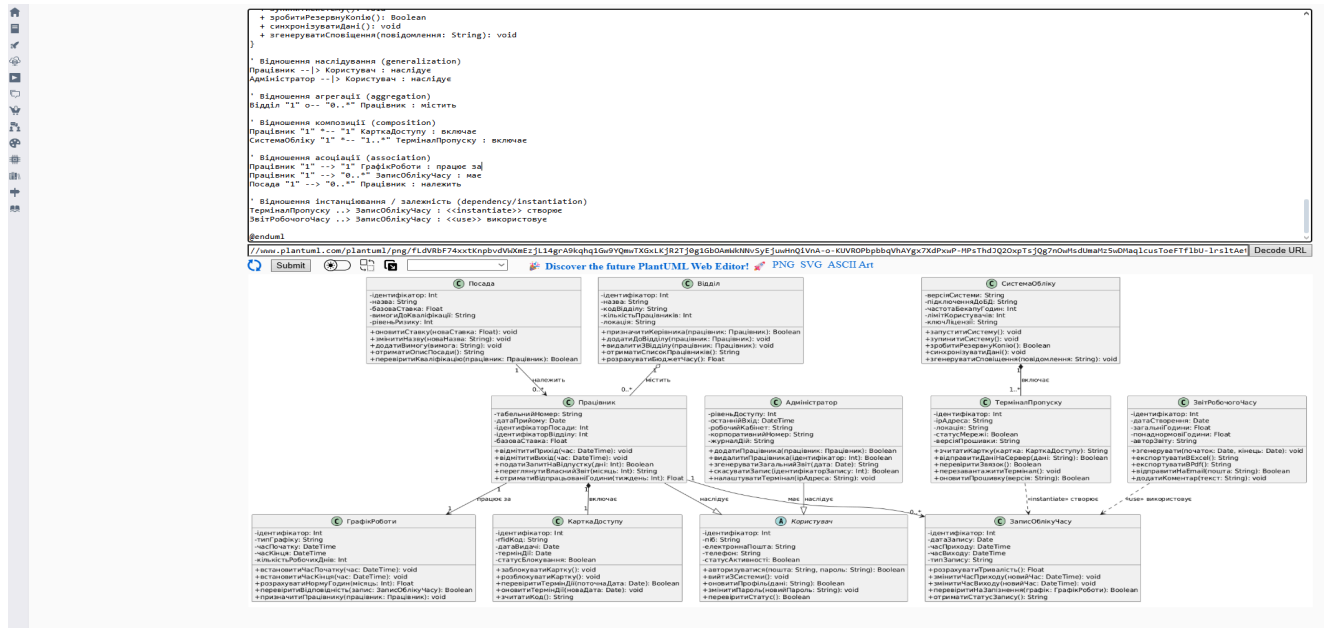


Рисунок 1 – Інтерфейс середовища PlantUML

Програмний код генерації діаграми

```
@startuml
```

```
skinparam classAttributeIconSize 0
```

```
abstract class Користувач {
```

- ідентифікатор: Int
- піб: String
- електроннаПошта: String
- телефон: String
- статусАктивності: Boolean

```
+ авторизоваться(почта: String, пароль: String): Boolean
```

```
+ вийтиЗСистеми(): void
```

```
+ оновитиПрофіль(дані: String): Boolean
```

```
+ змінитиПароль(новийПароль: String): void  
+ перевіритиСтатус(): Boolean  
}
```

```
class Працівник {  
    - табельнийНомер: String  
    - датаПрийому: Date  
    - ідентифікаторПосади: Int  
    - ідентифікаторВідділу: Int  
    - базоваСтавка: Float  
    + відмітитиПрихід(час: DateTime): void  
    + відмітитиВихід(час: DateTime): void  
    + податиЗапитНаВідпустку(дні: Int): Boolean  
    + переглянутиВласнийЗвіт(місяць: Int): String  
    + отриматиВідпрацьованіГодини(тиждень: Int): Float  
}
```

```
class Адміністратор {  
    - рівеньДоступу: Int  
    - останнійВхід: DateTime  
    - робочийКабінет: String  
    - корпоративнийНомер: String  
    - журналДій: String  
    + додатиПрацівника(працівник: Працівник): Boolean  
    + видалитиПрацівника(ідентифікатор: Int): Boolean  
    + згенеруватиЗагальнийЗвіт(дата: Date): String
```

```
+ скасуватиЗапис(ідентифікаторЗапису: Int): Boolean
+ налаштуватиТермінал(ipАдреса: String): void
}
```

```
class Відділ {
    - ідентифікатор: Int
    - назва: String
    - кодВідділу: String
    - кількістьПрацівників: Int
    - локація: String
    + призначитиКерівника(працівник: Працівник): Boolean
    + додатиДоВідділу(працівник: Працівник): void
    + видалитиЗВідділу(працівник: Працівник): void
    + отриматиСписокПрацівників(): String
    + розрахуватиБюджетЧасу(): Float
}
```

```
class Посада {
    - ідентифікатор: Int
    - назва: String
    - базоваСтавка: Float
    - вимогиДоКваліфікації: String
    - рівеньРизику: Int
    + оновитиСтавку(новаСтавка: Float): void
    + змінитиНазву(новаНазва: String): void
    + додатиВимогу(вимога: String): void
}
```

```
+ отриматиОписПосади(): String  
+ перевіритиКваліфікацію(працівник: Працівник): Boolean  
}
```

```
class ЗаписОблікуЧасу {  
    - ідентифікатор: Int  
    - датаЗапису: Date  
    - часПриходу: DateTime  
    - часВиходу: DateTime  
    - типЗапису: String  
    + розрахуватиТривалість(): Float  
    + змінитиЧасПриходу(новийЧас: DateTime): void  
    + змінитиЧасВиходу(новийЧас: DateTime): void  
    + перевіритиНаЗапізнення(графік: ГрафікРоботи): Boolean  
    + отриматиСтатусЗапису(): String  
}
```

```
class ГрафікРоботи {  
    - ідентифікатор: Int  
    - типГрафіку: String  
    - часПочатку: DateTime  
    - часКінця: DateTime  
    - кількістьРобочихДнів: Int  
    + встановитиЧасПочатку(час: DateTime): void  
    + встановитиЧасКінця(час: DateTime): void  
    + розрахуватиНормуГодин(місяць: Int): Float
```

```
+ перевіритиВідповідність(запис: ЗаписОблікуЧасу): Boolean
+ призначитиПрацівнику(працівник: Працівник): void
}
```

```
class КарткаДоступу {
    - ідентифікатор: Int
    - rfidКод: String
    - датаВидачі: Date
    - термінДії: Date
    - статусБлокування: Boolean
    + заблокуватиКартку(): void
    + розблокуватиКартку(): void
    + перевіритиТермінДії(поточнаДата: Date): Boolean
    + оновитиТермінДії(новаДата: Date): void
    + зчитатиКод(): String
}
```

```
class ТерміналПропуску {
    - ідентифікатор: Int
    - ipАдреса: String
    - локація: String
    - статусМережі: Boolean
    - версіяПрошивки: String
    + зчитатиКартку(картка: КарткаДоступу): String
    + відправитиДаніНаСервер(дані: String): Boolean
    + перевіритиЗв'язок(): Boolean
}
```

```
+ перезавантажитиТермінал(): void  
+ оновитиПрошивку(версія: String): Boolean  
}
```

```
class ЗвітРобочогоЧасу {  
    - ідентифікатор: Int  
    - датаСтворення: Date  
    - загальніГодини: Float  
    - понаднормовіГодини: Float  
    - авторЗвіту: String  
    + згенерувати(початок: Date, кінець: Date): void  
    + експортуватиВExcel(): String  
    + експортуватиВPdf(): String  
    + відправитиНаEmail(пошта: String): Boolean  
    + додатиКоментар(текст: String): void  
}
```

```
class СистемаОбліку {  
    - версіяСистеми: String  
    - підключенняДоБД: String  
    - частотаБекапуГодин: Int  
    - лімітКористувачів: Int  
    - ключЛіцензії: String  
    + запуститиСистему(): void  
    + зупинитиСистему(): void  
    + зробитиРезервнуКопію(): Boolean
```



```
+ синхронізуватиДані(): void
+ згенеруватиСповідання(повідомлення: String): void
}
```

' Відношення наслідування (generalization)

Працівник --|> Користувач : наслідує

Адміністратор --|> Користувач : наслідує

' Відношення агрегації (aggregation)

Відділ "1" o-- "0..*" Працівник : містить

' Відношення композиції (composition)

Працівник "1" *-- "1" КарткаДоступу : включає

СистемаОбліку "1" *-- "1..*" ТерміналПропуску : включає

' Відношення асоціації (association)

Працівник "1" --> "1" ГрафікРоботи : працює за

Працівник "1" --> "0..*" ЗаписОблікуЧасу : має

Посада "1" --> "0..*" Працівник : належить

' Відношення інстанціювання / залежність (dependency/instantiation)

ТерміналПропуску ..> ЗаписОблікуЧасу : <<instantiate>> створює

ЗвітРобочогоЧасу ..> ЗаписОблікуЧасу : <<use>> використовує

@endum1

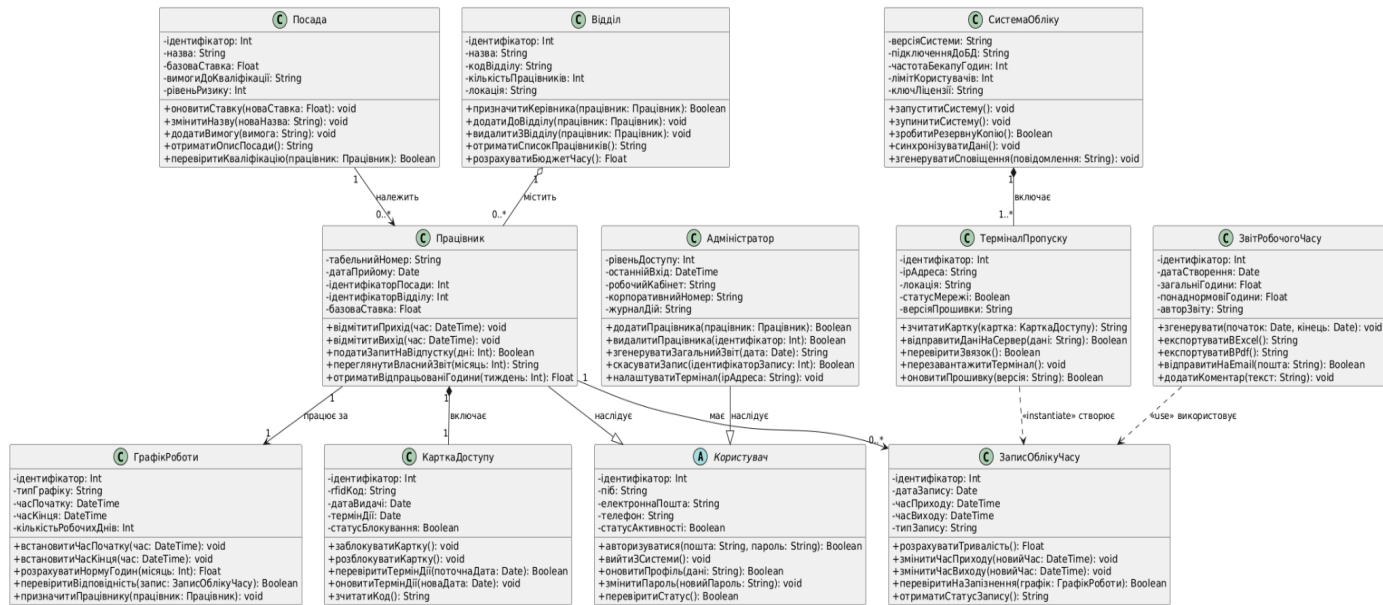


Рисунок 2 – Фінальна діаграма класів системи обліку робочого часу

Висновок: Під час виконання лабораторної роботи було здобуто практичні навички побудови діаграм класів (Class Diagram) мовою UML. Для заданої предметної області (Варіант 25) було спроектовано структуру з 11 класів, деталізовано їх атрибути та операції. На діаграмі успішно реалізовано та відображено всі основні типи відношень між об'єктами (асоціацію, агрегацію, композицію, наслідування та залежність), що повністю відповідає вимогам завдання.